SIMATS ENGINEERING

ASSESSMENT TOOL 3 – Assignment 2

COURSE NAME: STATISTICS WITH R PROGRAMMING **COURSE CODE: ITA04**

COURSE OUTCOME COVERED

CO3: *Construct and manipulate data frames using reshaping, merging, and file I/O operations in R to prepare structured datasets for analysis.. (BL3, BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks: 100**

Annexure

Assignment 2

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Assignment 2:

- **Generate 5 Questions for your Capstone Project module**
- **Apply the formulae and use the relevant graph to visualize it**
- **Upload the file in Github link**
- **Save the 5 Questions and Gihub link in the Excell sheet**

CAPSTONE PROJECT · R PROGRAMMING

AquaGuard-R — Module 04

Interactive R Shiny Dashboard: five questions, with a graph for each

Source: AquaGuard-R_Module2_Module4.pptx, slides 4–6 (Module 04 mind map, architecture, and the Module 02 → Module 04 hand-off).

About this document

Module 04 of AquaGuard-R is the Shiny dashboard — the layer that makes the cleaned data of Module 1, the statistics of Module 2 and the trained model of Module 3 usable by someone who does not write R. Its five tabs are Home, Water Quality, Statistics, Prediction and Risk Map, built with shiny, shinydashboard, ggplot2, plotly, leaflet and DT. The five questions below take one tab each; each answer is followed by the R code behind it and a graph of the panel it produces.

A note on the data: the figures were generated in R with ggplot2 from a reproducible 420-sample dataset simulated for this document, classified Low / Moderate / High by the same BIS IS 10500:2012 rule Module 02 uses. They show the form of each dashboard panel and are illustrative — the live dashboard draws the same panels from the project dataset, with plotly and leaflet in place of static images.

Question 1 — Home tab

What does the Home tab of the AquaGuard-R dashboard display, and how is the contamination risk of the whole dataset summarised there?

Answer

The Home tab is the landing screen of the dashboard. It answers the question “how bad is the water overall?” before the user filters anything. Branch 01 of the Module 04 mind map lists what it carries: the total number of samples, the number of locations covered, the average value of each water quality parameter, and the distribution of contamination risk.

These appear as the summary value boxes of the architecture diagram, drawn with shinydashboard::valueBox(). The Low / Moderate / High labels behind them are not computed in the dashboard — they are produced by Module 02 from the BIS IS 10500:2012 acceptable and permissible limits, and Module 04 only colours and counts them. This is the first of the four hand-offs on the Module 02 → Module 04 slide.

- Inputs: cleaned data from Module 1 and risk labels from Module 2, loaded once when the app starts.
- Widgets: valueBox() for the counts, a plotly bar chart for the risk mix.
- Purpose: give a one-glance verdict on the whole dataset.

R code

```
# --- Home tab: value boxes and the risk distribution chart -------------
output$box_samples <- renderValueBox(
  valueBox(nrow(wq), "Samples analysed", icon = icon("flask")))


output$box_high <- renderValueBox(
  valueBox(sum(wq$risk == "High"), "High risk samples", color = "red"))


output$risk_plot <- renderPlotly({
  d <- as.data.frame(table(wq$risk)); names(d) <- c("risk", "n")
  p <- ggplot(d, aes(risk, n, fill = risk)) +
         geom_col(width = 0.55) +
         geom_text(aes(label = n), vjust = -0.6) +
         scale_fill_manual(values = c(Low      = "#0ca30c",
                                      Moderate = "#fab219",
                                      High     = "#d03b3b")) +
         labs(x = NULL, y = "Number of samples")
  ggplotly(p)
})
```

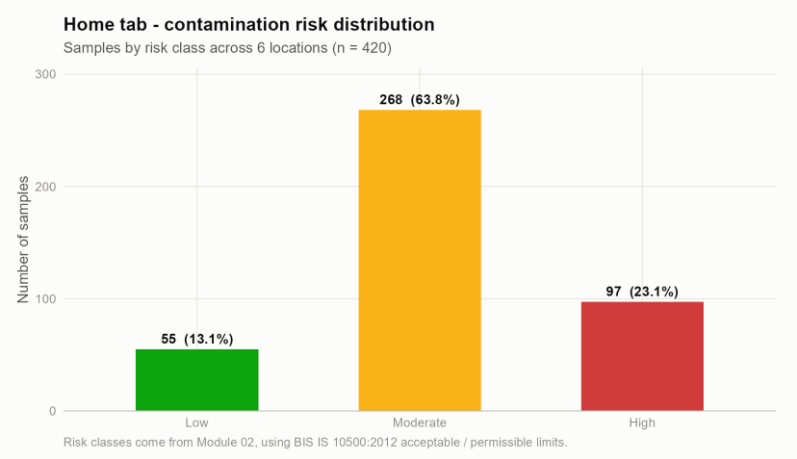


Figure 1 — Home tab: how the 420 samples split across the three risk classes.

How to read it: most samples (63.8%) are Moderate — above the acceptable limit on at least one parameter but still within the permissible limit. The 23.1% marked High are the samples that breach a permissible limit, and they are what the Risk Map tab will put on the map in red.

Question 2 — Water Quality tab

How does the Water Quality tab let a user compare a parameter across locations, and what does such a comparison reveal?

Answer

Branch 02 of the mind map gives this tab three filters — location, parameter and date. The sidebar collects them, and the server wraps the data in a reactive() expression so that every chart on the tab redraws the moment any filter changes; nothing has to be recomputed by hand. The charts themselves are plotly objects, so the user can hover a box to read the exact median and quartiles, or zoom into one location.

The comparison that matters most is a single parameter plotted against location, with the BIS acceptable limit drawn as a reference line. That turns a table of numbers into a ranking: the wards whose boxes sit above the line are the ones to investigate first.

- Filters: selectInput() for location and parameter, dateRangeInput() for the period.
- Reactivity: reactive() holds the filtered data; renderPlotly() redraws the panel.
- Reference line: the BIS acceptable limit for the selected parameter.

R code

```
# --- Water Quality tab: filter, then redraw -----------------------------
filtered <- reactive({
  wq %>% filter(location %in% input$loc,
                date >= input$dates[1], date <= input$dates[2])
})

output$param_plot <- renderPlotly({
  p <- ggplot(filtered(), aes(location, .data[[input$param]])) +
         geom_hline(yintercept = bis_limit[[input$param]],
                    linetype = "22", colour = "#898781") +
         geom_boxplot(fill = "#2a78d6", colour = "#184f95", width = 0.5) +
         labs(x = NULL, y = input$param)
  ggplotly(p)
})
```

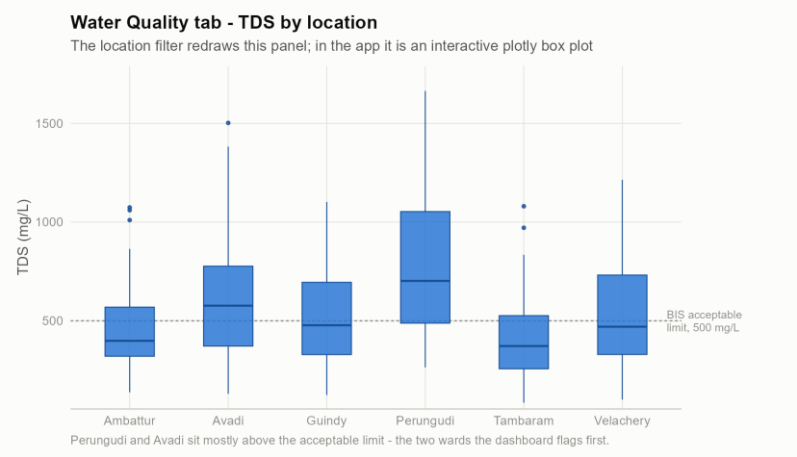


Figure 2 — Water Quality tab: TDS compared across the six locations, against the BIS acceptable limit.

How to read it: the thick line in each box is the median, the box covers the middle half of the samples and the dots are outliers. Perungudi and Avadi are the only two locations whose median sits above 500 mg/L, so the dashboard flags them first; Tambaram is the cleanest.

Question 3 — Statistics tab

How are the Module 02 statistical results carried into the Statistics tab, and how is the correlation heatmap read?

Answer

The Statistics tab is where Module 02 becomes visible. Branch 03 lists its three contents, and the Module 02 → Module 04 slide names the source of each one: the Pearson / Spearman correlation matrix becomes the heatmap, the t-test, ANOVA and Chi-square p-values become a results table rendered with DT, and the parameters that the logistic regression found important become the input fields of the Prediction tab.

The heatmap is read by colour and by number together. Only the lower triangle is drawn, because a correlation matrix is symmetric and the upper half would repeat the same values. Each cell also prints its r, so the chart never asks the reader to judge a value from colour alone.

- Heatmap: cor() on the numeric parameters, drawn with geom_tile().
- Results table: the hypothesis test output from Module 02, shown with DT::datatable().
- Rule of thumb: |r| above about 0.5 is a strong relationship, near 0 is none.

R code

```
# --- Statistics tab: Module 02 results on screen -----------------------
params <- c("pH", "TDS", "turbidity", "nitrate", "hardness", "DO")
cm <- cor(wq[params], method = "pearson")
cm[upper.tri(cm)] <- NA                    # keep the lower triangle only
d  <- as.data.frame(as.table(cm)); names(d) <- c("row", "col", "r")

output$corr_plot <- renderPlot({
  ggplot(na.omit(d), aes(col, row, fill = r)) +
    geom_tile(colour = "#fcfcfb", linewidth = 1.1) +
    geom_text(aes(label = sprintf("%.2f", r))) +
    scale_fill_gradient2(low = "#2a78d6", mid = "#f0efec",
                         high = "#e34948", midpoint = 0, limits = c(-1, 1))
})

output$test_table <- renderDT(datatable(module2_tests))   # t-test / ANOVA p-values
```

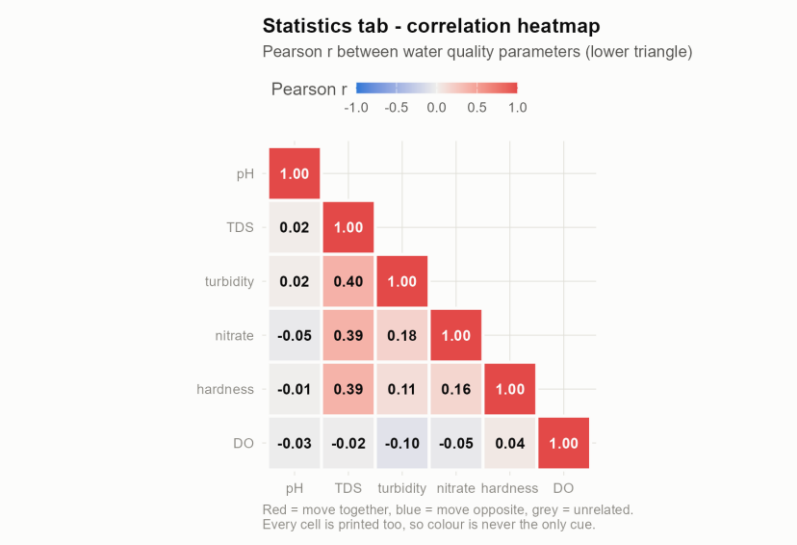


Figure 3 — Statistics tab: Pearson correlation between the six water quality parameters.

How to read it: red means the two parameters rise together, blue means one rises as the other falls, and grey means they are unrelated. TDS is the hub — it moves with turbidity (0.40), nitrate (0.39) and hardness (0.39), which is why it is the parameter the other tabs lead with. pH is independent of everything, and DO moves very slightly the other way.

Question 4 — Prediction tab

How does the Prediction tab turn the values a user types into a Low / Moderate / High risk class?

Answer

Branch 04 describes the tab in one line: enter pH, TDS, turbidity and nitrate, and get back a risk class from the best model. The prediction flow at the bottom of the architecture slide spells out the four steps — the user types the readings into numericInput() boxes, presses the Predict button, the server calls predict(best_model, newdata) and a coloured risk badge appears.

The model itself is not trained inside the app. Module 3 trains several models, keeps the best one and saves it as an .rds file; the dashboard loads that file once when it starts, which is why a prediction is instant. The four input fields are exactly the parameters Module 02 found significant, so the three modules stay consistent with one another.

- Inputs: numericInput() for pH, TDS, turbidity and nitrate; actionButton() to submit.
- Model: readRDS() of the best model saved by Module 3, loaded once at startup.
- Output: predict() returns the class (or a probability), shown as a coloured badge.

R code

```
# --- Prediction tab: user input -> risk class --------------------------
best_model <- readRDS("models/best_model.rds")     # saved by Module 3

observeEvent(input$predict, {
  newdata <- data.frame(pH        = input$pH,
                        TDS       = input$TDS,
                        turbidity = input$turbidity,
                        nitrate   = input$nitrate)

  cls <- predict(best_model, newdata = newdata, type = "class")

  # with the illustrative logistic model below, a probability is returned
  # instead and the class is decided at the 0.50 threshold:
  # p <- predict(m, newdata, type = "response"); cls <- if (p >= 0.5) "High" else "Low"

  output$risk_badge <- renderUI(
    span(class = paste0("badge-", tolower(cls)), paste(cls, "risk")))
})
```

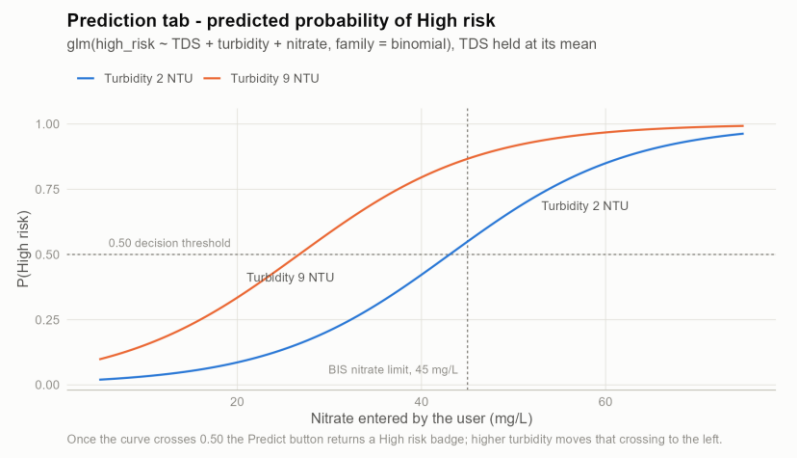


Figure 4 — Prediction tab: how the fitted model's probability of High risk responds to the values entered.

How to read it: each curve is the model's probability of High risk as the user raises the nitrate value, holding TDS at its mean. Where a curve crosses 0.50 the badge flips from Low to High. At 2 NTU turbidity that happens at about 43 mg/L of nitrate — close to the BIS limit of 45 — but at 9 NTU it happens at about 26 mg/L, because dirty water needs less nitrate to be judged unsafe.

Question 5 — Risk Map tab

What does the Risk Map tab display, and what does the dashboard do when the dataset has no latitude and longitude?

Answer

Branch 05 pairs leaflet with OpenStreetMap tiles and places one colour-coded marker per sampling point — green for Low, amber for Moderate, red for High, using the same labels the Home tab counts. Clicking a marker opens a popup with the location name and its risk class, which is what makes the dashboard usable by someone who has never seen the dataset: the map answers “where” and “how risky” at the same time.

The map depends on coordinates, and not every water quality dataset carries them. The design therefore includes a fallback, noted on the mind map slide: if the data has no latitude and longitude columns, the tab drops the Leaflet map and shows location-wise charts instead. The information is the same — which ward is worst — only the geography is missing.

- Map: leaflet() + addTiles() + addCircleMarkers(), coloured by colorFactor().
- Colours: the Low / Moderate / High labels produced by Module 02.
- Fallback: a location-wise stacked bar chart when lat / lon are absent.

R code

```
# --- Risk Map tab: leaflet markers, with a fallback --------------------
pal <- colorFactor(c("#0ca30c", "#fab219", "#d03b3b"),
                   levels = c("Low", "Moderate", "High"))


output$risk_map <- renderLeaflet({
  req(all(c("lat", "lon") %in% names(wq)))
  leaflet(filtered()) %>% addTiles() %>%
    addCircleMarkers(~lon, ~lat, color = ~pal(risk), radius = 7,
                     popup = ~paste0(location, ": ", risk, " risk"))
})


output$risk_fallback <- renderPlotly({        # no coordinates in the data
  d <- count(wq, location, risk) %>% group_by(location) %>%
         mutate(share = 100 * n / sum(n))
  ggplotly(ggplot(d, aes(share, location, fill = risk)) +
             geom_col(width = 0.6, colour = "#fcfcfb") +
             scale_fill_manual(values = c(Low      = "#0ca30c",
                                          Moderate = "#fab219",
                                          High     = "#d03b3b")))
})
```

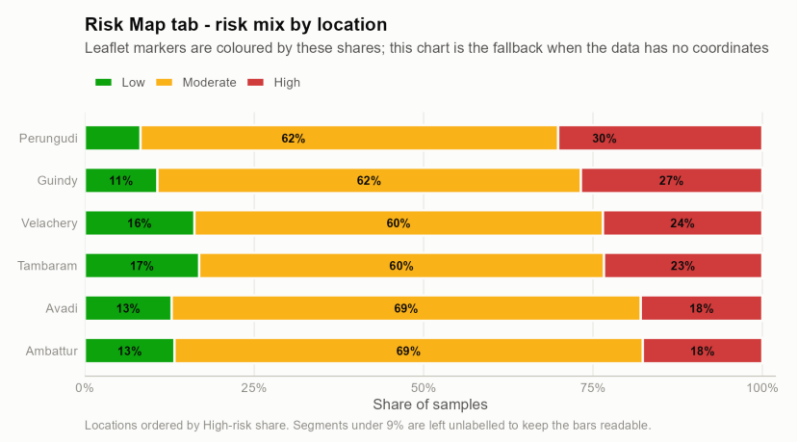


Figure 5 — Risk Map tab: the risk mix of each location, which is also the fallback view when the data has no coordinates.

How to read it: each bar is one location split into its three risk shares, ordered with the worst at the top. Perungudi has the largest High share at 30% and Guindy follows at 27%, while Ambattur and Avadi are lowest at 18% — on the map, the same mix of marker colours.

Appendix — common setup used by all five figures

Every figure in this document was produced from the same simulated dataset and the same ggplot2 theme. The code below is the shared setup; the per-question code blocks run on top of it.

```
library(ggplot2); library(dplyr)

set.seed(2026)
n   <- 420
loc <- c("Ambattur", "Avadi", "Guindy", "Perungudi", "Tambaram", "Velachery")
wq  <- data.frame(
  location  = factor(sample(loc, n, TRUE), levels = loc),
  pH        = round(rnorm(n, 7.2, 0.55), 2),
  TDS       = round(pmax(90, rgamma(n, 4.2, scale = 110)), 0),
  turbidity = round(pmax(0.2, rgamma(n, 2.1, scale = 0.75)), 2),
  nitrate   = round(pmax(1, rnorm(n, 26, 14)), 1),
  hardness  = round(pmax(40, rnorm(n, 185, 70)), 0),
  DO        = round(pmax(2, rnorm(n, 6.4, 1.1)), 1))

# parameters that share a source move together
wq <- transform(wq,
  nitrate   = round(pmin(120, nitrate  + 0.030 * (TDS - mean(TDS))), 1),
  hardness  = round(pmin(900, hardness + 0.180 * (TDS - mean(TDS))), 0),
  turbidity = round(pmax(0.2, turbidity + 0.0016 * (TDS - mean(TDS))), 2))

# Module 02 risk classification (BIS IS 10500:2012)
beyond_permissible <- with(wq, TDS > 2000 | turbidity > 5 | nitrate > 45 |
                               hardness > 600 | pH < 6.5 | pH > 8.5)
above_acceptable   <- with(wq, TDS > 500 | turbidity > 1 | nitrate > 45 |
                               hardness > 200)
wq$risk <- factor(ifelse(beyond_permissible, "High",
                  ifelse(above_acceptable, "Moderate", "Low")),
                  levels = c("Low", "Moderate", "High"))

table(wq$risk)        # Low 55   Moderate 268   High 97
```

Takeaway from the closing slide: Module 02 explains why water is risky; Module 04 shows anyone where and how risky it is.